

Descriptive Statistics II

Spread, Shape, Z-Scores & Correlation — with Python

CodeByPaxto: The AI & ML Playground

Hafiz Waseem Ahmed (Paxto)



What We're Covering Today

- 1 Measures of spread — range, variance, standard deviation, and IQR
- 2 Standard deviation vs. standard error — two very different questions
- 3 Z-scores — measuring exactly how unusual a value really is
- 4 The shape of a distribution — skewness, kurtosis, and outliers
- 5 Correlation — the actual coefficient behind last session's heatmap
- 6 Today's tasks and a mini project — a full descriptive profile of real data

PART 1

Measures of Spread / Dispersion

Two datasets can share the exact same mean and still look nothing alike — spread is what central tendency alone can't tell you



What Is Spread, and Why Does It Matter?



Same center, different shape

- Spread describes how tightly or loosely your data clusters around its center — exactly what a measure of central tendency can never tell you on its own
- [18, 20, 22] and [2, 20, 38] both have a mean of 20 — but describing them with just "the average is 20" hides a completely different reality
- Spread is what tells you whether a value is genuinely typical, or actually unusual — a foundation this entire session builds on
- Every measure on the next few slides answers the same underlying question in a different way: "how far, on average, do values stray from the center?"



Same Mean, Different Spread

- Both lists average exactly 50 — the mean cannot tell them apart.
- The standard deviation can: A is tight, B is spread out.
- `np.std()` is the one call; Part 1 builds it by hand first.

```
import numpy as np

a = [48, 49, 50, 51, 52]
b = [10, 30, 50, 70, 90]

print(np.mean(a), np.mean(b))    # 50.0 50.0
print(round(np.std(a), 2))       # 1.41
print(round(np.std(b), 2))       # 28.28
```



Range — The Simplest Measure of Spread

- Range is simply the distance between the largest and smallest value in a dataset — the fastest possible way to measure spread
- It only looks at two values, and completely ignores everything in between — a single extreme outlier can inflate it dramatically
- Useful as a quick first glance, but rarely the measure you'd actually report or rely on for a real analysis



```
range = max(x) - min(x)
```

```
# Example: [12, 15, 18, 20, 55]
```

```
range = 55 - 12 = 43
```

```
# One extreme value (55) dominates  
# the entire result — range tells you  
# nothing about the other four values
```



Range in Code

- `max()` and `min()` are built into Python — no library needed.
- `np.ptp` (“peak to peak”) is numpy's one-call version.
- Watch one outlier swallow the result.

```
data = [12, 15, 18, 20, 55]

print(max(data) - min(data))  # 43

import numpy as np
print(np.ptp(data))          # 43

clean = [12, 15, 18, 20]
print(max(clean) - min(clean))  # 8
```



Variance — The Average Squared Distance From the Mean

- Variance measures spread by finding how far every single value is from the mean, squaring each distance, then averaging those squared distances
- Squaring serves two purposes: it makes every distance positive (so they don't cancel out), and it punishes bigger deviations disproportionately more
- The cost is that variance's units are squared too — "dollars squared" or "years squared" don't mean anything intuitive on their own



```
variance = (1/n) · Σ (xi – mean)2
```

```
# Example: [2, 4, 6], mean = 4
```

```
# deviations: -2, 0, 2
```

```
# -> squared: 4, 0, 4
```

```
variance = (4 + 0 + 4) / 3 = 2.67
```



Variance from Scratch

- Follow the slide exactly: deviations, then squares, then the average.
- Pure Python — every step is visible.
- `statistics.pvariance` gives the same answer as a check.

```
data = [2, 4, 6]

mean = sum(data) / len(data)
deviations = [x - mean for x in data]
squared = [d ** 2 for d in deviations]
variance = sum(squared) / len(data)

print(deviations)      # [-2.0, 0.0, 2.0]
print(squared)         # [4.0, 0.0, 4.0]
print(round(variance, 2)) # 2.67

import statistics
print(round(statistics.pvariance(data), 2)) # 2.67
```



Standard Deviation — Undoing the Squaring

- Standard deviation is simply the square root of variance — it exists specifically to undo variance's squared units, bringing spread back into the original, meaningful scale
- "The standard deviation of exam scores is 8 points" is something you can actually reason about; "the variance is 64 points squared" isn't
- This is the single most commonly reported measure of spread in real statistics — you'll see it constantly from this point forward



```
standard deviation =  $\sqrt{\text{variance}}$ 
```

```
# Continuing the example:
```

```
# variance = 2.67
```

```
std_dev =  $\sqrt{2.67} \approx 1.63$ 
```

```
# Back in the original units —
```

```
# values typically sit about 1.63
```

```
# away from the mean of 4
```



Standard Deviation in Code

- Standard deviation is just `math.sqrt` of the variance.
- `numpy`, `statistics` and `pandas` all offer it directly.
- Careful: they do not all divide the same way — next slide.

```
import math
import statistics
import numpy as np

data = [2, 4, 6]
variance = statistics.pvariance(data)

print(round(math.sqrt(variance), 2)) # 1.63
print(round(statistics.pstdev(data), 2)) # 1.63
print(round(np.std(data), 2)) # 1.63
```



Population vs. Sample: The ddof Gotcha

- The slides divide by n — that is the population formula.
- For a sample, statistics divides by $n - 1$ (Bessel's correction) to avoid underestimating spread.
- numpy defaults to n , pandas defaults to $n - 1$. Always check which one you got.

```
import numpy as np
import pandas as pd
import statistics

data = [2, 4, 6]

print(round(np.std(data), 2))    # 1.63
print(round(np.std(data, ddof=1), 2)) # 2.0
print(round(pd.Series(data).std(), 2)) # 2.0
print(round(statistics.stdev(data), 2)) # 2.0
```



Interquartile Range (IQR) — The Middle 50%

Q

Spread, without the outliers

- IQR measures the spread of the middle 50% of your data.
- It is calculated as $Q3 - Q1$, where $Q1$ is the 25th percentile and $Q3$ is the 75th percentile.
- Because IQR focuses only on the middle 50%, it ignores the extreme values at both ends. This makes it much less affected by outliers than the range and standard deviation.
- This makes IQR especially useful when your data is skewed or contains outliers.
- Rule of thumb: skewed data or data with outliers → report IQR alongside the median. Roughly symmetric, clean data → standard deviation alongside the mean is often appropriate.



IQR in Code

- `np.percentile` finds Q1 and Q3; the IQR is the difference.
- `scipy.stats.iqr` does it in one call.
- The outlier barely moves the IQR but wrecks the standard deviation.

```
import numpy as np
from scipy import stats

data = [10, 12, 14, 15, 16, 18, 20, 95]

q1, q3 = np.percentile(data, [25, 75])
print(q3 - q1)           # 5.0
print(stats.iqr(data))   # 5.0

print(round(np.std(data), 1)) # 26.6
print(round(np.std(data[:-1]), 1)) # 3.2
```



PART 1



Practice Set — Measures of Spread

Try these yourself first — pencil, paper or a quick Python cell.

- 1 Calculate the range, variance, and standard deviation of [5, 7, 7, 9, 12] by hand
- 2 Explain why variance is rarely reported directly, using what you know about its units
- 3 For a dataset with one extreme outlier, explain which measure of spread would be least affected, and why
- 4 Two classes have the same average test score, but Class A has a standard deviation of 3 and Class B has 15. Explain what this tells you about each class

PART 2

Standard Deviation vs. Standard Error

Two measures that sound alike, get confused constantly, and answer genuinely different questions



What Is Standard Error?



±

Uncertainty in the estimate

- Standard error measures how much a sample statistic (like a sample mean) would likely bounce around if you repeated the sampling process over and over
- It answers a question standard deviation was never built to answer: "if I took a different sample from the same population, how different might my mean turn out?"
- Standard error shrinks as your sample size grows — more data gives you a more stable, trustworthy estimate of the true population mean
- This concept is the direct seed of confidence intervals, which get fully covered in Session 21



Standard Deviation vs. Standard Error

Standard Deviation

- Describes spread WITHIN your actual data
- "How far, typically, does one data point sit from the mean?"
- Does NOT shrink just because you collect more data — it reflects real variability in the population

Standard Error

- Describes uncertainty in an ESTIMATE (like a sample mean)
- "How much would my sample mean likely shift if I resampled?"
- DOES shrink as sample size grows — more data means a more precise estimate



Calculating Standard Error

- Standard error is calculated directly from your standard deviation and your sample size — no new data collection needed
- Dividing by the square root of n means standard error shrinks slowly — quadrupling your sample size only halves your standard error
- This relationship is exactly why "collect more data" has real, predictable, diminishing returns on precision



standard error = standard deviation / $\sqrt{n}$

Example: std_dev = 10, n = 25

$SE = 10 / \sqrt{25} = 10 / 5 = 2$

Quadruple the sample to n = 100:

$SE = 10 / \sqrt{100} = 10 / 10 = 1$

only halved, not quartered



Standard Error in Code

- SE = standard deviation / $\sqrt{n}$ — straight from the slide.
- `scipy.stats.sem` does it for you, using the sample $(n - 1)$ SD.
- Quadruple n and the SE only halves.

```
import math
import numpy as np
from scipy import stats

std_dev = 10
print(std_dev / math.sqrt(25)) # 2.0
print(std_dev / math.sqrt(100)) # 1.0

sample = [52, 48, 55, 50, 47, 53]
print(round(stats.sem(sample), 2)) # 1.25
print(round(np.std(sample, ddof=1) / math.sqrt(6), 2))
# → 1.25
```



SD Stays, SE Shrinks

- Same population, growing samples.
- The sample SD hovers near 10 forever — it describes real variability.
- The SE keeps falling — our estimate of the mean gets more precise.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(7)
population = rng.normal(100, 10, 100_000)

def sd_se(n):
    s = rng.choice(population, n)
    return (round(float(s.std(ddof=1)), 1),
            round(float(stats.sem(s)), 2))

print(sd_se(25))          # (8.8, 1.77)
print(sd_se(100))         # (10.2, 1.02)
print(sd_se(400))         # (9.2, 0.46)
```



Practice Set — SD vs. SE

Try these yourself first — pencil, paper or a quick Python cell.

- 1 Explain, in your own words, why standard error shrinks with a larger sample but standard deviation does not
- 2 Calculate the standard error for a sample of 64 values with a standard deviation of 16
- 3 A news report says "average height: 170cm" without mentioning sample size. Explain why you can't evaluate how precise that estimate is without more information
- 4 Explain why a researcher reporting results from an entire population (not a sample) would never need to calculate a standard error

PART 3

Z-Score

One number that answers "how unusual is this, really?" — on any scale, for any variable



What Is a Z-Score?

Z

**Distance, measured in std
devs**

- A z-score tells you exactly how many standard deviations a value sits above or below the mean
- A z-score of 0 means "exactly average." Positive means above average, negative means below — and the size tells you how far
- This process is called standardizing — it converts any value, from any scale or unit, into this one universal "how unusual is this" number
- A z-score beyond ± 2 is generally considered noticeably unusual; beyond ± 3 is genuinely rare



Calculating a Z-Score

- Subtract the mean from your value, then divide by the standard deviation — that's the entire calculation
- The subtraction centers the value around 0; the division rescales it in units of "how many standard deviations"
- A negative result simply means the value sits below the mean — not an error, just direction

```
z = (x - mean) / standard deviation
```

```
# Example: exam mean = 70, std_dev = 8
```

```
# Your score: 86
```

```
z = (86 - 70) / 8  
    = 16 / 8  
    = 2.0
```

```
# 2 standard deviations above average
```



Z-Scores in Code

- One line by hand: $(x - \text{mean}) / \text{std_dev}$.
- `scipy.stats.zscore` converts a whole list at once.
- The mean's own z-score is always 0.

```
import numpy as np
from scipy import stats

z = (86 - 70) / 8
print(z)                                # 2.0

scores = [60, 65, 70, 75, 80]
print(stats.zscore(scores).round(2))
# → [-1.41 -0.71  0.    0.71  1.41]
print(round(np.mean(stats.zscore(scores)), 2)) # 0.0
```



A Z-Score, Visualized



This data point sits 1.5 standard deviations above the mean — noticeably above average, but not extreme.



Real-World Use: Comparing Apples to Oranges

- Z-scores let you fairly compare two values that come from completely different scales — exactly the kind of comparison raw numbers can't make honestly
- A student who scored 86 on a test averaging 70 (std dev 8), and 650 on a different test averaging 500 (std dev 100) — which result was actually stronger?
- Converting both to z-scores answers this immediately and fairly

```
# Test A: mean 70, std_dev 8, score 86
z_A = (86 - 70) / 8 = 2.0

# Test B: mean 500, std_dev 100, score 650
z_B = (650 - 500) / 100 = 1.5

# Test A was the stronger relative
# performance – 2.0 standard deviations
# above average beats 1.5
```



Comparing Apples to Oranges

- Two different tests, two different scales — raw scores cannot be compared.
- A small function turns each into a z-score.
- Test A wins even though 650 is a bigger number than 86.

```
def z_score(x, mean, sd):  
    return (x - mean) / sd  
  
z_a = z_score(86, 70, 8)  
z_b = z_score(650, 500, 100)  
  
print(z_a)           # 2.0  
print(z_b)           # 1.5  
print("A" if z_a > z_b else "B") # A
```



The Empirical Rule — 68 / 95 / 99.7

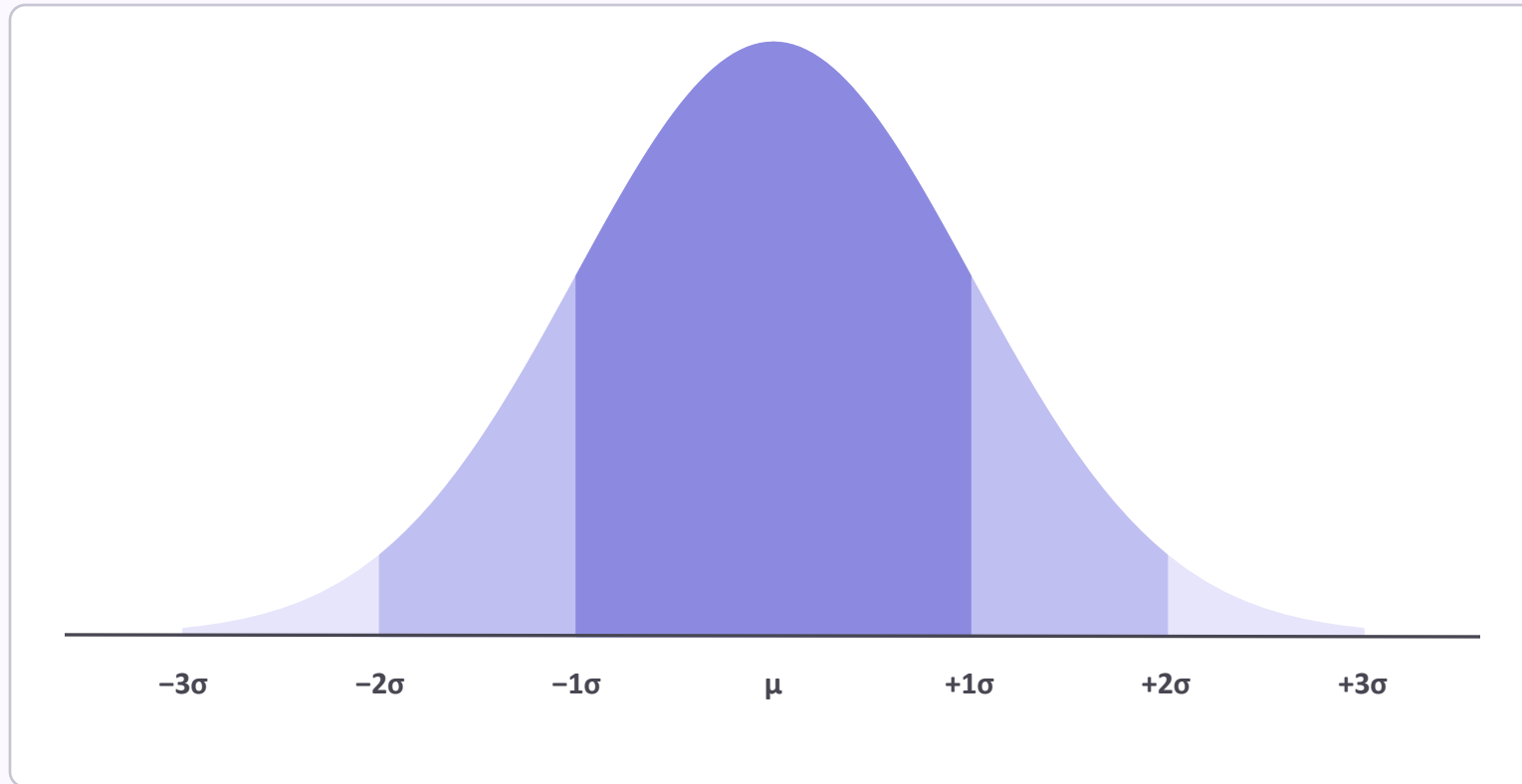
%

**What a z-score actually
covers**

- For roughly normal (bell-shaped) data, z-scores translate directly into percentages of your data — this is exactly why z-scores are so useful for judging "how unusual"
- About 68.2% of values fall within 1 standard deviation of the mean (z between -1 and +1)
- About 95.4% fall within 2 standard deviations, and about 99.7% fall within 3
- This means a z-score beyond ± 2 already puts you outside roughly 95% of your data — and beyond ± 3 is genuinely rare, under half a percent of values



The Empirical Rule — 68 / 95 / 99.7



68.2%

within $\pm 1\sigma$



95.4%

within $\pm 2\sigma$



99.7%

within $\pm 3\sigma$

This only holds when your data is roughly normally distributed — Session 20 covers exactly why the normal distribution has this property.



Checking the Empirical Rule

- Simulate 100,000 normal values, then count how many fall within ± 1 , 2, 3 SDs.
- `stats.norm.cdf` gives the exact theory numbers.
- The two agree — this is the 68 / 95 / 99.7 rule.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(1)
x = rng.normal(0, 1, 100_000)

def within(k):
    return round(float(np.mean(np.abs(x) < k)), 3)

print(within(1))      # 0.684
print(within(2))      # 0.955
print(within(3))      # 0.997

print(round(stats.norm.cdf(2) - stats.norm.cdf(-2), 3))
# → 0.954
```



PART 3



Practice Set — Z-Scores

Try these yourself first — pencil, paper or a quick Python cell.

- 1 Calculate the z-score for a value of 45, from a dataset with mean 60 and standard deviation 10
- 2 Explain in your own words what a z-score of -2.5 means, without using the word "standard deviation" in your explanation
- 3 Using the empirical rule, estimate roughly what percentage of values fall ABOVE a z-score of +2
- 4 Two employees are rated on completely different scoring systems at their two companies. Explain how z-scores would let you compare them fairly

PART 4

Shape of a Distribution

Two datasets can share a mean AND a standard deviation, and still look completely different once you see their shape



Frequency Distributions & Histograms, Revisited

#

**Counting how often, not
just what**

- A frequency distribution shows how often each value, or range of values, occurs in your dataset — the histogram from Session 18 is simply this idea, drawn as bars
- The overall shape traced by those bars is exactly what this part of the session is about — not just what the chart looks like, but what that shape tells you statistically
- The same mean and standard deviation can hide wildly different shapes — shape is genuinely its own, separate piece of information
- This shape directly determines which statistical tests are even valid to use later, in Sessions 21 and 22



Symmetric vs. Skewed Distributions



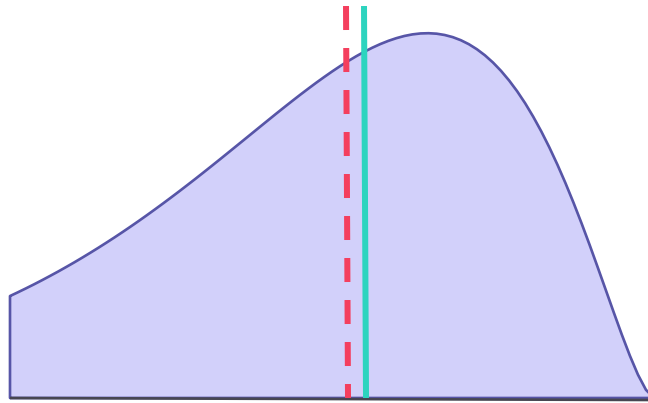
Balanced, or leaning one way

- A symmetric distribution looks roughly the same on both sides of its center — mean and median sit close together
- A skewed distribution has a longer tail stretching out on one side — pulling the mean toward that tail, while the median resists and stays closer to the bulk of the data
- Right-skewed (positive skew): a long tail toward higher values — classic example: income, where a few very high earners stretch the tail right
- Left-skewed (negative skew): a long tail toward lower values — classic example: age at retirement, where a few very early retirements stretch the tail left



Symmetric vs. Skewed Distributions

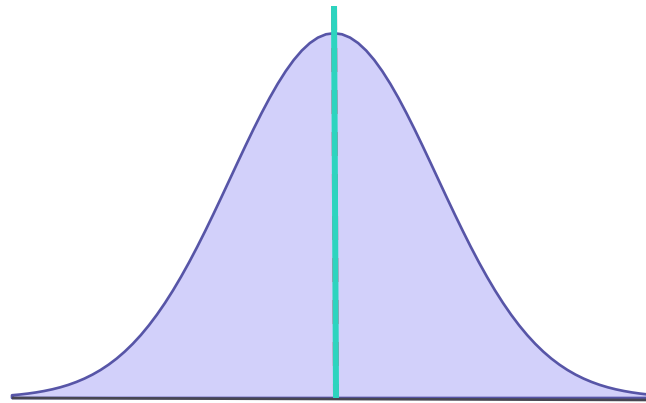
Left-Skewed (Negative)



● Mean

● Median

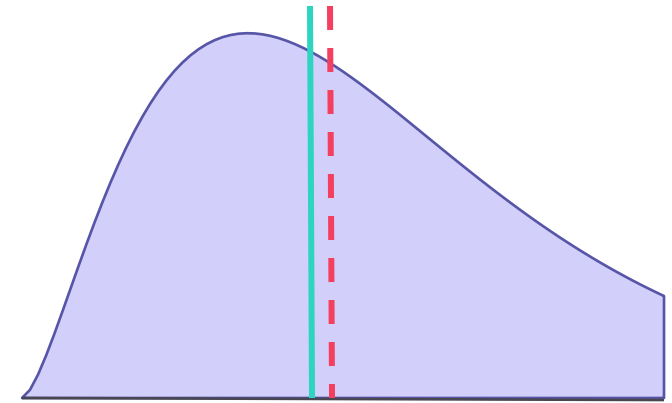
Symmetric



● Mean

● Median

Right-Skewed (Positive)



● Mean

● Median

The tail pulls the mean toward it, while the median resists — this is exactly why a skewed distribution makes the mean and median disagree, and why the median is the safer choice when skew is present.



Skew Pulls the Mean

- Incomes are the classic right-skewed data: a few huge values form the tail.
- The mean gets dragged toward the tail; the median resists.
- Mean above median is the quick signature of right skew.

```
import numpy as np

rng = np.random.default_rng(2)
income = rng.lognormal(mean=10.5, sigma=0.8, size=5000)

print(round(income.mean()))    # 49981
print(round(np.median(income))) # 36640
print(income.mean() > np.median(income)) # True
```



Skewness vs. Kurtosis

- Skewness measures asymmetry — exactly the left-vs-right lean covered on the last slide, now given a number: 0 is symmetric, positive leans right, negative leans left
- Kurtosis measures tail weight — how much of your data lives in extreme values far from the mean, compared to a normal distribution
- High kurtosis means heavier tails and more extreme outliers than you'd expect; low kurtosis means a flatter distribution with fewer extremes
- Neither needs to be calculated by hand in practice — the goal here is recognizing what these words mean when a tool reports them to you



Skewness: rough intuition

skew > 0 -> tail stretches right

skew < 0 -> tail stretches left

skew ≈ 0 -> roughly symmetric

Kurtosis: rough intuition

kurtosis high -> heavy tails,
more extreme outliers

kurtosis low -> light tails, flatter



Skewness & Kurtosis in Code

- `scipy.stats.skew` and `kurtosis` return the numbers the slide describes.
- Normal data is near 0 for both (scipy reports excess kurtosis).
- Right-skewed, heavy-tailed data scores clearly positive.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(3)
normal = rng.normal(0, 1, 10_000)
skewed = rng.exponential(1, 10_000)

print(abs(stats.skew(normal)) < 0.1) # True
print(round(stats.skew(skewed), 2)) # 2.02
print(round(stats.kurtosis(normal), 2)) # -0.06
print(round(stats.kurtosis(skewed), 2)) # 6.43
```



Outliers via the IQR Rule



A precise, repeatable fence

- Session 18 introduced outliers conceptually through the box plot — this is the actual rule that decides exactly where the whiskers end
- The rule builds a "fence" 1.5 times the IQR beyond Q1 and Q3 — anything outside that fence is flagged as an outlier
- This gives you a precise, repeatable, and consistent way to flag outliers, instead of relying on a subjective "that number looks big" judgment
- Not every outlier is an error — some are perfectly real, important extreme values. Flagging is the first step; deciding what to do about it always requires judgment



The 1.5×IQR Fence Rule in Action

- Calculate IQR, then build a lower fence and an upper fence exactly 1.5×IQR beyond Q1 and Q3
- Anything below the lower fence, or above the upper fence, gets flagged as an outlier
- This is exactly the rule box plot whiskers use to decide where to stop — anything beyond them is typically plotted as an individual point



```
IQR = Q3 - Q1  
lower_fence = Q1 - 1.5 × IQR  
upper_fence = Q3 + 1.5 × IQR
```

```
# Example: Q1 = 20, Q3 = 50  
# IQR = 30  
# lower = 20 - 45 = -25  
# upper = 50 + 45 = 95  
# Anything outside is an outlier
```



The 1.5×IQR Fence in Code

- Wrap the rule in a small function you can reuse on any list.
- Q1 and Q3 come from `np.percentile`; the fences sit $1.5 \times \text{IQR}$ beyond them.
- Anything outside the fences is flagged as an outlier.

```
import numpy as np

def iqr_outliers(data):
    q1, q3 = np.percentile(data, [25, 75])
    iqr = q3 - q1
    lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    return lo, hi, [x for x in data if x < lo or x > hi]

data = [18, 20, 22, 30, 38, 50, 52, 120]
lo, hi, out = iqr_outliers(data)

print(lo, hi)           # -22.0 94.0
print(out)              # [120]
```



Practice Set — Shape of a Distribution

Try these yourself first — pencil, paper or a quick Python cell.

- 1 For a dataset of household incomes, predict whether it's more likely left-skewed or right-skewed, and explain your reasoning
- 2 Explain in your own words why a skewed distribution makes the mean less trustworthy as a "typical value"
- 3 Given $Q1 = 10$ and $Q3 = 40$, calculate the lower and upper fences using the $1.5 \times IQR$ rule
- 4 Find a real histogram (news article, report, or dataset) and describe its shape — symmetric, left-skewed, or right-skewed

PART 5

Correlation

The heatmap from Session 18 was the picture — this is the actual number behind it



What Does Correlation Actually Measure?



Do they move together?

- Correlation measures the strength and direction of a straight-line relationship between two numerical variables
- Positive correlation: as one variable increases, the other tends to increase too. Negative correlation: as one increases, the other tends to decrease
- Correlation says absolutely nothing about WHY two things move together — only THAT they do, and how consistently
- This is precisely the statistic behind every cell of the correlation heatmap from Session 18 — now you'll see exactly how each number is actually calculated



The Pearson Correlation Coefficient

- The Pearson correlation coefficient, r , is the standard measure of straight-line correlation — the exact number that fills every cell of a correlation heatmap
- r always falls between -1 and $+1$ — the sign gives direction, the size gives strength
- $r = 0$ doesn't necessarily mean "no relationship at all" — it means no LINEAR relationship. A strong curved relationship can still produce an r near 0



r compares how x and y vary together

against how much each varies alone

$r = +1.0$ perfect positive line

$r = +0.7$ strong positive

$r = 0.0$ no linear relationship

$r = -0.7$ strong negative

$r = -1.0$ perfect negative line



Pearson r in Code

- `np.corrcoef` returns a 2x2 matrix; the off-diagonal cell is `r`.
- `scipy.stats.pearsonr` gives `r` and a `p-value` (`p-values` arrive in Inferential Statistics).
- A perfect curve like $y = x^2$ still gives `r` near 0 — `r` only sees straight lines.

```
import numpy as np
from scipy import stats

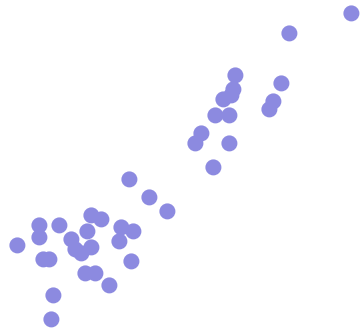
hours = [1, 2, 3, 4, 5, 6]
score = [52, 55, 61, 64, 70, 74]

print(round(np.corrcoef(hours, score)[0, 1], 3))
# → 0.996
r, p = stats.pearsonr(hours, score)
print(round(r, 3))          # 0.996

x = [-3, -2, -1, 0, 1, 2, 3]
y = [v ** 2 for v in x]     # perfect curve
print(round(np.corrcoef(x, y)[0, 1], 2)) # 0.0
```

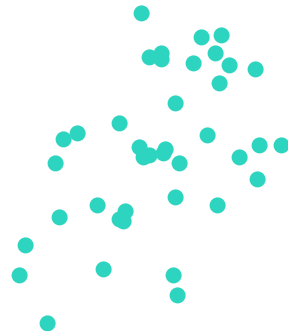


Correlation Strength, at a Glance



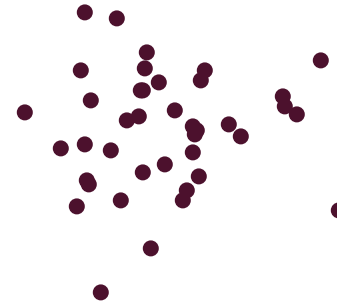
Strong Positive

$r \approx +0.95$



Weak Positive

$r \approx +0.35$



No Correlation

$r \approx 0.00$



Strong Negative

$r \approx -0.92$

r ranges from -1 to $+1$. The sign tells you the direction (do they rise together, or does one fall as the other rises?). The magnitude — how close to ± 1 — tells you how tightly the points hug a straight line. Near 0 means no meaningful straight-line relationship, not necessarily "no relationship at all."



Correlation $\neq$ Causation

$\neq$

**Moving together, not
causing**

- Two variables can correlate strongly without either one causing the other at all — this is one of the most important cautions in all of statistics
- Classic example: ice cream sales and drowning incidents both rise together in summer — neither causes the other. A third factor, hot weather, drives both
- A strong correlation is a genuinely useful signal to investigate further — it's just never, on its own, proof of a cause-and-effect relationship
- Proving causation requires a controlled experiment, or careful methods specifically designed to rule out other explanations — far beyond what a correlation coefficient alone can tell you



Correlation $\neq$ Causation: A Hidden Third Factor

- Heat drives BOTH ice cream sales and swimming accidents.
- Sales and accidents never affect each other, yet they correlate strongly.
- Only the hidden factor — temperature — explains both.

```
import numpy as np

rng = np.random.default_rng(5)
temp = rng.normal(25, 6, 300)
ice_cream = 3 * temp + rng.normal(0, 8, 300)
accidents = 0.5 * temp + rng.normal(0, 3, 300)

r = np.corrcoef(ice_cream, accidents)[0, 1]
print(round(r, 2))          # 0.6

r_t = np.corrcoef(temp, ice_cream)[0, 1]
print(round(r_t, 2))        # 0.91
```



Practice Set — Correlation

Try these yourself first — pencil, paper or a quick Python cell.

- 1 A dataset shows $r = 0.85$ between hours studied and exam score. Describe the direction and strength of this relationship in plain English
- 2 Find a real headline claiming one thing "causes" another, and explain whether the evidence described is actually causal or just correlational
- 3 Explain why $r = 0.05$ between two variables doesn't necessarily mean they're unrelated
- 4 Propose a plausible hidden third factor for a correlation between "number of firefighters at a fire" and "amount of damage caused"



Value Ranges at a Glance — Spread Measures

1

Range

0 to $+\infty$. Always non-negative
— the distance between two values can't be negative

2

Variance

0 to $+\infty$. Always non-negative
— every deviation is squared before averaging, so it can never go below 0

3

Standard Deviation

0 to $+\infty$. Always non-negative
— the square root of variance is never negative. Exactly 0 means every value is identical

4

Standard Error

0 to $+\infty$. Always non-negative
— shrinks toward 0 as sample size grows, but never becomes negative



Value Ranges at a Glance — Position & Relationship Measures

1

Z-Score

$-\infty$ to $+\infty$. In practice, almost always falls between -3 and $+3$ for real data

2

Correlation (r)

-1 to $+1$ exactly. -1 and $+1$ are perfect relationships; 0 means no linear relationship

3

Skewness

Unbounded in theory, but real data usually falls roughly between -3 and $+3$. 0 means symmetric

4

Kurtosis

Unbounded in theory. "Excess kurtosis" is centered at 0 for a perfectly normal distribution



A Full Profile in a Few Lines

- pandas describe() gives count, mean, std and the quartiles at once.
- Add skew and the IQR to complete the picture.
- This is the pattern for today's mini project.

```
import numpy as np
import pandas as pd
from scipy import stats

df = pd.DataFrame({"salary": [40, 42, 45, 48, 50, 52, 55, 60, 150]})
col = df["salary"]

print(col.describe().round(1)) # count, mean, std, quartiles
print(round(stats.skew(col), 2)) # 2.32
print(stats.iqr(col)) # 10.0
```



Where This Is Headed



**Description becomes a real
test**

- Every tool in this session — mean, standard deviation, z-score — was used to describe data you already fully have in front of you
- Very soon, these exact same tools get reused for a very different job: deciding whether a difference you've observed in a SAMPLE is likely real in the full population, or just random noise
- A t-test, which you'll meet properly in Session 22, is built almost entirely from a mean, a standard deviation, and a standard error — nothing you haven't just learned
- Session 20 fills in exactly one missing piece first: probability — the language needed to say precisely what "likely real" and "just random noise" actually mean

INDUSTRY-STYLE TASKS

Build the Toolkit Every Real Analysis Depends On

Not abstract formula drills — spread, shape, and correlation are what turn a raw dataset into an honest, defensible description



TASKS



Part A: Spread & SD vs. SE

Small tasks — do them in order.

- 1 Calculate the range, variance, and standard deviation of a real numerical column from a dataset of your choice.
- 2 For that same column, calculate the standard error assuming your data is a sample, and explain what it tells you that standard deviation doesn't.
- 3 Explain, using your own numbers, why quadrupling your sample size only halves your standard error rather than quartering it.



TASKS



Part B: Z-Scores & Distribution Shape

Small tasks — do them in order.

- 1 Calculate z-scores for the five highest and five lowest values in a real numerical column, and identify any that exceed ± 2 .
- 2 Plot a histogram of a real numerical column and describe its shape — symmetric, left-skewed, or right-skewed — and explain your reasoning.
- 3 Apply the $1.5 \times \text{IQR}$ rule to that same column and list any values it flags as outliers.



TASKS



Part C: Correlation

Small tasks — do them in order.

- 1 Choose two numerical columns from a real dataset and calculate their Pearson correlation coefficient.
- 2 Create a scatter plot of those two variables, and describe whether the visual matches what the coefficient suggested.
- 3 Propose one plausible reason two variables might correlate without one causing the other, using your own chosen columns as the example.



TASKS



Part D: Applying Everything to Real Data

Small tasks — do them in order.

- 1 Take a full numerical column and report its mean, median, standard deviation, and shape (skew direction) in one short paragraph.
- 2 Identify one pair of columns worth checking for correlation, and explain why you suspect a relationship before calculating it.
- 3 Write a two-sentence "descriptive summary" of your dataset suitable for someone who has never seen it, using only what you learned in Sessions 18 and 19.

MINI PROJECT

Put It All Together

A complete descriptive profile of a real dataset, spread and shape included



Mini Project: Full Descriptive Profile of a Real Dataset



Σ

**Every skill from today,
combined**

- Choose any real, numerical-heavy dataset, and for its key column calculate: range, variance, standard deviation, and standard error
- Identify the column's shape (symmetric or skewed), and flag any outliers using the $1.5 \times \text{IQR}$ rule
- Calculate z-scores for at least three specific values, and explain in plain English how unusual each one really is
- Choose two numerical columns, calculate their correlation, and write a short, honest note on whether a real causal relationship seems plausible or unlikely



Mini Project Starter

- Swap in your own dataset and column — everything else stays the same.
- Range, variance, SD, SE, skew and outliers in one reusable block.
- Finish with a correlation between two columns.

```
import numpy as np
import pandas as pd
from scipy import stats

df = pd.DataFrame({"hours": [1, 2, 3, 4, 5, 6, 7, 8],
                  "score": [50, 55, 52, 63, 68, 72, 75, 98]})
x = df["score"]

print(x.max() - x.min(), round(x.std(), 1)) # 48 15.7
print(round(stats.sem(x), 2)) # 5.55
print(round(stats.skew(x), 2)) # 0.88
print(round(df["hours"].corr(x), 2)) # 0.93
```

RECAP

Know Your Data First

A mean alone is a rumor. Spread, shape, and correlation are what turn it into an honest description of your data. Next up:
Probability